

# Exigências do Projeto Final

## Mineração de Dados Aplicada com CRISP-DM

IF1014 - Tópicos Avançados em Sistemas de Informação 5  
Centro de Informática - UFPE  
Prof. Leandro Maciel Almeida

Semestre 2026.1

### Marcos do projeto:

Checkpoint do projeto: **20/05/2026**.

Apresentação dos resultados: **27/05/2026**.

Entrega final do relatório e dos slides: **10/06/2026**.

## Sumário

|           |                                                     |          |
|-----------|-----------------------------------------------------|----------|
| <b>1</b>  | <b>Identificação da disciplina</b>                  | <b>1</b> |
| <b>2</b>  | <b>Cenário empresarial simulado</b>                 | <b>1</b> |
| 2.1       | Papéis sugeridos no grupo . . . . .                 | 1        |
| 2.2       | Briefing do cliente (modelo) . . . . .              | 2        |
| <b>3</b>  | <b>Objetivos de aprendizagem</b>                    | <b>2</b> |
| <b>4</b>  | <b>Requisitos da base de dados</b>                  | <b>2</b> |
| <b>5</b>  | <b>Partição treino e teste (regra inegociável)</b>  | <b>3</b> |
| <b>6</b>  | <b>Algoritmos obrigatórios</b>                      | <b>3</b> |
| <b>7</b>  | <b>Workflow experimental obrigatório</b>            | <b>4</b> |
| <b>8</b>  | <b>Busca de hiperparâmetros</b>                     | <b>4</b> |
| <b>9</b>  | <b>Gráficos exigidos</b>                            | <b>5</b> |
| <b>10</b> | <b>Justificativa experimental</b>                   | <b>5</b> |
| <b>11</b> | <b>Formato de entrega</b>                           | <b>6</b> |
| <b>12</b> | <b>Cronograma e marcos</b>                          | <b>6</b> |
| <b>13</b> | <b>Apresentação final</b>                           | <b>6</b> |
| <b>14</b> | <b>Integridade acadêmica e uso de IA generativa</b> | <b>7</b> |
| <b>15</b> | <b>Glossário técnico</b>                            | <b>7</b> |

# 1 Identificação da disciplina

A disciplina **IF1014 - Tópicos Avançados em Sistemas de Informação 5**, sob o tema *Mineração de Dados Aplicada com CRISP-DM*, integra a grade do curso de Sistemas de Informação do Centro de Informática da UFPE. O projeto final descrito neste documento é a principal entrega da disciplina e percorre o ciclo completo da metodologia CRISP-DM aplicado a dados reais.

**Objetivo do projeto.** Conduzir, em grupo, um projeto de mineração de dados que cubra todas as fases do CRISP-DM: *Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation* e *Deployment*. O grupo deve aplicar dez algoritmos de classificação, conduzir busca de hiperparâmetros para cada um, comparar variantes do conjunto de treino e selecionar o melhor modelo a partir de evidências experimentais.

## 2 Cenário empresarial simulado

O projeto simula um cenário realista de empresa: cada grupo atende a um **cliente fictício** que precisa de uma solução baseada em dados. Toda decisão técnica deve ser comunicada e justificada como em um cenário profissional.

### 2.1 Papéis sugeridos no grupo

- **Cientista de dados:** responsável por modelagem, busca de hiperparâmetros e análise estatística das comparações.
- **Engenheiro de dados:** responsável pela carga, pelo split treino e teste, pelos pipelines de pré-processamento e pela reprodutibilidade do experimento.
- **Comunicador:** responsável pela narrativa do relatório e dos slides, pela tradução das métricas em linguagem acessível ao cliente e pelo briefing.

Os papéis não são excludentes; cada integrante deve participar das três frentes, mas é esperado que cada um lidere uma delas durante a execução.

### 2.2 Briefing do cliente (modelo)

O grupo deve preencher e versionar um documento curto com o briefing do seu cliente fictício. O modelo abaixo deve aparecer no início do relatório final.

*O cliente [nome fictício] atua no setor [setor] e precisa de um modelo capaz de [tarefa de classificação], em que a saída pertença a uma das classes [lista de classes]. O sucesso será medido por [métrica principal] sobre o conjunto de teste, com restrição de [tempo de inferência, custo de erro, etc.]. As implicações éticas relevantes são [viés, privacidade, impacto social].*

|                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Tarefa do grupo:</b> Adapte o briefing ao seu dataset, indicando claramente o problema, a métrica principal, as restrições do cliente e os riscos éticos. Esse briefing orienta as decisões do projeto inteiro.</p> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3 Objetivos de aprendizagem

Ao concluir o projeto, o grupo deve ser capaz de:

1. Modelar um problema real seguindo as seis fases do CRISP-DM.
2. Executar análise exploratória e justificar decisões de pré-processamento com dados.
3. Aplicar dez algoritmos de classificação com busca de hiperparâmetros e comparação experimental.
4. Discutir o trade-off entre complexidade do modelo e generalização usando curvas de treino vs validação.
5. Avaliar e comparar modelos com testes estatísticos pareados (Wilcoxon ou paired t-test).
6. Comunicar resultados técnicos a um público com perfil de cliente, em linguagem clara, apoiada em gráficos e tabelas.

### 4 Requisitos da base de dados

A base escolhida pelo grupo deve atender simultaneamente:

- Formato **tabular** (linhas como instâncias, colunas como atributos).
- Volume superior a **50.000 instâncias**.
- Tarefa de **classificação multiclasse** (variável alvo com mais de 2 classes).
- Documentação pública que permita citar a fonte no relatório.

**Repositórios sugeridos.** Kaggle, UCI Machine Learning Repository, OpenML, Hugging Face Datasets.

**Bases candidatas (exemplos).** Forest Cover Type (581k instâncias, 7 classes); Letter Recognition expandido; Poker Hand (1M instâncias, 10 classes); KDD Cup 99 (múltiplas classes de ataque); Census Income segmentado por faixa; Statlog Shuttle (7 classes). A escolha deve ser justificada no briefing do cliente.

### 5 Partição treino e teste (regra inegociável)

- O conjunto bruto deve ser dividido em **treino** e **teste logo no início** do projeto, com split estratificado e **random\_state** fixo.
- Toda a fase de *Modeling* (preparação, busca de hiperparâmetros, baseline, variantes, comparações) usa **apenas o treino**, com **validação cruzada** interna.
- O conjunto de teste é **tocado uma única vez**, na atividade final, com os melhores modelos já selecionados.

**Por que essa regra existe.** Tocar o teste durante a busca de hiperparâmetros causa *data leakage*: o modelo passa a ser otimizado para o teste, perdendo-se a estimativa honesta de generalização. Em um cenário real isso significa entregar ao cliente um modelo otimista que falha em produção.

**Guarda anti-leakage.** O repositório template implementa uma guarda no `data_loader.py`: a função `load_test(unlock_token)` só retorna o teste se o aluno passar o token `I_AM_IN_FINAL_EVALUATION`. Esse token deve aparecer apenas na seção de avaliação final do notebook. Violações da regra (ler o teste antes da hora) são penalizadas na rubrica.

## 6 Algoritmos obrigatórios

O projeto deve aplicar os dez algoritmos abaixo. A divisão em duas partes reflete a sequência das aulas e não limita o uso de variantes adicionais.

### Parte 1.

- K-NN (*K-Nearest Neighbors*).
- LVQ (*Learning Vector Quantization*), via pacote `sklvq`.
- Árvore de Decisão (CART, critérios Gini e Entropia).
- SVM (*Support Vector Machine*).
- Random Forest.

### Parte 2.

- MLP (*Multilayer Perceptron*).
- Comitê de Redes Neurais (votação ou bagging de MLPs).
- Stacking heterogêneo (combina classificadores das partes 1 e 2 com um meta-modelo).
- XGBoost.
- LightGBM.

## 7 Workflow experimental obrigatório

O fluxo de experimentos do projeto segue a sequência abaixo. As etapas em destaque correspondem ao foco da rubrica.

1. **Carga** da base bruta a partir do arquivo original.
2. **Split estratificado** em treino e teste, com `random_state` fixo. O teste é isolado por uma guarda anti-leakage.
3. **EDA** (análise exploratória) feita estritamente sobre o treino.
4. **Preparação baseline:** pré-processamento mínimo necessário para os modelos rodarem (imputação trivial e codificação básica de categóricos), sem decisões de balanceamento ou engenharia de características.
5. **Busca de hiperparâmetros** para cada um dos dez algoritmos sobre o treino com validação cruzada. Para cada configuração testada, registrar a curva de desempenho de *treino vs validação*. **[foco da rubrica]**
6. **Baseline:** consolidar os dez melhores modelos da etapa anterior. Esse é o ponto de comparação para todas as próximas variações.

7. **Variantes do conjunto de treino:** aplicar técnicas de balanceamento (SMOTE, under-sampling, *class weights*), escalas (StandardScaler, MinMax, Robust), codificações (OneHot, Target, Ordinal) e engenharia de características (criação ou descarte de variáveis com base em evidência da EDA). **O teste não é tocado em nenhuma dessas operações.**
8. **Nova busca de hiperparâmetros** para os dez modelos em cada variante.
9. **Comparação com o baseline** usando teste pareado (Wilcoxon ou paired t-test) sobre as métricas obtidas por fold. **[foco da rubrica]**
10. **Seleção do melhor modelo final**, justificada por evidência estatística.
11. **Avaliação final única no conjunto de teste**, gerando relatório de classificação, matriz de confusão normalizada, ROC e PR macro multiclasse.

## 8 Busca de hiperparâmetros

A Tabela 1 traz *sugestões iniciais* de espaços de busca para os dez algoritmos. As faixas servem como ponto de partida; o grupo deve refinar em função das curvas de treino vs validação observadas em cada experimento.

### Recomendações práticas.

- Use `RandomizedSearchCV` em espaços grandes (SVM, MLP, XGBoost, LightGBM) e `GridSearchCV` apenas em espaços pequenos.
- Mantenha o número de *folds* consistente entre experimentos para que as comparações sejam pareadas (mesmos folds, mesmo `random_state`).
- Use `n_jobs=-1` com cuidado em máquinas com pouca memória; em Colab e Kaggle, prefira valores moderados (2 a 4).
- Persista `cv_results_` de cada busca como CSV para que o relatório possa retomar valores específicos.

## 9 Gráficos exigidos

- **Curva treino vs validação** para cada configuração testada, em todos os experimentos. Esse gráfico mostra o trade-off entre viés e variância e sustenta a escolha de hiperparâmetros.
- **Matriz de confusão normalizada** do melhor modelo de cada algoritmo (no baseline e na melhor variante).
- **Curvas ROC e PR macro multiclasse** dos modelos finais.
- **Gráfico de barras comparativo** de F1-macro entre baseline e cada variante.

## 10 Justificativa experimental

Toda decisão técnica do projeto deve ser justificada com dados experimentais. Decisões que se enquadram nesta regra incluem (mas não se limitam a): escolha de hiperparâmetros, técnica de balanceamento, codificação de categóricos, normalização de numéricos, criação ou descarte de variáveis, escolha do modelo final, escolha do meta-modelo no Stacking.

Tabela 1: Sugestões de espaços de busca para os dez algoritmos obrigatórios. Os valores são um ponto de partida e devem ser refinados pelos grupos a partir das curvas de treino vs validação observadas em cada experimento.

| Algoritmo            | Hiperparâmetros principais                                                                                                                                         | Faixa sugerida (ponto de partida)                                                                     | Estratégia              | Folds |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------|-------|
| K-NN                 | <code>n_neighbors</code> , <code>weights</code> , <code>metric</code>                                                                                              | 3 a 31 (ímpar), {uniform, distance}, {euclidean, manhattan, minkowski}                                | Grid                    | 5     |
| LVQ (sklvq)          | <code>distance_type</code> , <code>prototypes_per_class</code> , <code>activation_type</code>                                                                      | {squared-euclidean}, 1 a 5, {sigmoid, swish, identity}                                                | Grid                    | 5     |
| Árvore de Decisão    | <code>criterion</code> , <code>max_depth</code> , <code>min_samples_leaf</code> , <code>ccp_alpha</code>                                                           | {gini, entropy}, {None, 5 a 30}, 1 a 20, 0 a 0.05                                                     | Grid                    | 5     |
| SVM                  | <code>C</code> , <code>kernel</code> , <code>gamma</code>                                                                                                          | 0.01 a 100 (log), {linear, rbf, poly}, {scale, auto, 0.001 a 1}                                       | Randomized              | 5     |
| Random Forest        | <code>n_estimators</code> , <code>max_features</code> , <code>max_depth</code> , <code>min_samples_leaf</code>                                                     | 100 a 800, {sqrt, log2, 0.5}, {None, 10 a 30}, 1 a 10                                                 | Randomized              | 5     |
| MLP                  | <code>hidden_layer_sizes</code> , <code>activation</code> , <code>learning_rate_init</code> , <code>batch_size</code> , <code>alpha</code>                         | {(64,), (128,), (64,64), (128,64)}, {relu, tanh}, 1e-5 a 1e-1 (log), 1e-4 a 1e-1 (log), {32, 64, 128} | Randomized              | 5     |
| Comitê de RNAs       | <code>n_estimators</code> , <code>hidden_layer_sizes</code> , <code>base</code> , <code>voting</code>                                                              | 5 a 30, {(64,), (128,), (64,64)}, {soft, hard}                                                        | Randomized              | 5     |
| Stacking heterogêneo | <code>base</code> , <code>estimators</code> , <code>final_estimator</code> , <code>passthrough</code>                                                              | combos a partir do baseline (KNN, RF, SVM, MLP), {LogReg, MLP_pequena}, {True, False}                 | Grid em combos pequenos | 5     |
| XGBoost              | <code>n_estimators</code> , <code>max_depth</code> , <code>learning_rate</code> , <code>subsample</code> , <code>colsample_bytree</code> , <code>reg_lambda</code> | 200 a 1000, 3 a 12, 0.01 a 0.3 (log), 0.6 a 1.0, 0.6 a 1.0, 0.1 a 10 (log)                            | Randomized              | 5     |
| LightGBM             | <code>num_leaves</code> , <code>learning_rate</code> , <code>min_data_in_leaf</code> , <code>feature_fraction</code> , <code>bagging_fraction</code>               | 15 a 255, 0.01 a 0.3 (log), 5 a 100, 0.6 a 1.0, 0.6 a 1.0                                             | Randomized              | 5     |

## Padrão de argumentação esperado.

- Citar a métrica observada com **desvio entre folds** (ex.: F1-macro = 0,812 ± 0,014).
- Quando comparar duas opções ou comparar uma variante contra o baseline, aplicar **teste de Wilcoxon pareado** ou *paired t-test* sobre as métricas por fold e reportar o p-valor.
- Anexar a curva treino vs validação da configuração escolhida e do principal concorrente.
- Usar métricas robustas a desbalanceamento quando aplicável (F1-macro, balanced accuracy) em vez de acurácia simples.

## 11 Formato de entrega

- **Relatório final em PDF**, baseado no template fornecido (`relatorio.tex`). Limite recomendado de 25 páginas, sem incluir apêndices.

- **Repositório Git** com todo o código (notebook principal e módulos), preferencialmente *fork* ou clone do template oficial. O grupo deve adicionar o professor como colaborador no repositório privado.
- **Slides em PDF**, baseados no template Beamer fornecido (`slides.tex`).
- **Logs de experimentos** (CSV de `cv_results_` ou *runs* de MLflow) versionados ou linkados a partir do relatório.

## 12 Cronograma e marcos

| Data       | Atividade                                                |
|------------|----------------------------------------------------------|
| 20/05/2026 | Checkpoint do projeto (estado parcial dos experimentos). |
| 27/05/2026 | Apresentação dos resultados (15 minutos cronometrados).  |
| 10/06/2026 | Entrega final do relatório e dos slides.                 |

## 13 Apresentação final

- **Duração:** 15 minutos cronometrados, com interrupção se exceder.
- **Formato:** slides clean, com bullets, figuras e tabelas. Não incluir código.
- **Participação:** todos os integrantes do grupo apresentam parte do trabalho. A presença é verificada na data agendada.
- **Estrutura:** uma seção por fase do CRISP-DM, com *ênfase reforçada* em busca de hiperparâmetros (Modeling) e em comparação entre baseline e melhorias (Evaluation).
- **Material entregue:** PDF dos slides até a véspera da apresentação.

## 14 Integridade acadêmica e uso de IA generativa

- É permitido o uso de assistentes de IA generativa para apoio na escrita, na exploração de bibliotecas e na revisão de código, desde que o grupo **compreenda** o que entrega e **declare** o uso no relatório (seção final de *Reprodutibilidade e ferramentas*).
- Plágio de código, cópia direta de relatórios de terceiros ou apresentação de resultados que não podem ser reproduzidos a partir do repositório implicam sanções disciplinares conforme as normas da UFPE.
- O grupo deve garantir que o repositório reproduz os resultados do relatório com no máximo um comando para instalar dependências e o notebook principal executando de ponta a ponta.

## 15 Glossário técnico

### Baseline

Conjunto inicial de melhores modelos obtidos com pré-processamento mínimo. Ponto de comparação para todas as variantes posteriores.

### Curva treino vs validação

Gráfico que compara o desempenho do modelo no próprio treino e no fold de validação em função de um hiperparâmetro ou da iteração. Útil para diagnosticar overfitting e underfitting.

**Data leakage**

Vazamento de informação do conjunto de teste (ou do futuro) para o treino. Inflama as métricas de validação e produz modelos otimistas que falham em produção.

**Holdout estratificado**

Partição do conjunto bruto em treino e teste preservando a proporção das classes em cada subconjunto.

**Hiperparâmetro**

Configuração do algoritmo que não é aprendida a partir dos dados (ex.: `n_neighbors` no K-NN, `max_depth` na árvore). Definida via busca.

**Validação cruzada**

Técnica em que o treino é dividido em  $k$  *folds*; o modelo treina em  $k - 1$  folds e valida no fold restante, repetindo  $k$  vezes. Reduz a variância da estimativa de desempenho.

**F1-macro**

Média simples do F1 calculado por classe. Robusta a desbalanceamento.

**Wilcoxon pareado**

Teste estatístico não paramétrico que compara duas configurações a partir de medidas pareadas (mesmos folds), útil para decidir se uma variante supera o baseline.